

Model-View-Controller

This is the most basic architecture you can possibly imagine. I think we all know this one, but let me share my thoughts on it. In this architecture, we spread the business logic into two main layers: models and controllers. Usually, there are also some additional layers. We'll talk about them later.

By using only controllers and model the main idea is this:

- Models contain the business logic
- Controllers contain the "glue" code

However, in the real world, controllers also contain business code. Let's say we're working on a real estate listing site. It has only a few basic features such as:

- Listing CRUD
- A listing can be scheduled to be published at some point in the future. We store this date in the `publish_at` column.
- An admin must accept a listing before it gets published. Each listing must go through a review/moderation process. This date is stored in the `accepted_at` column.

Please notice that the `accepted_at` is in the past tense but the `publish_at` is in the present. The reason is that the `publish_at` date can be in the future but also in the past. If it's in the future it means that the listing is scheduled to be published on that day.

This is a pretty basic example application with only one model. To confuse you I'll only use locations and prices from my country because I have no idea how the rest of the world works (and also there's a chance you Google these cities and see pictures such as [these](#)):

id	title	location	price	published	accepted	publish_at	accepted_at
1	Listing #1	Budapest	44900000	false	false	NULL	NULL
2	Listing #2	Keszthely	28990000	true	true	2023-02-18 13:43:00	2023-02-18 13:30:32
3	Listing #3	Debrecen	24990000	false	true	2023-06-01 19:00:00	2023-05-12 14:54:27

You can also use a `status` column to store a listing's status but in this example, I'm gonna use two bool columns.

CRUD

This is what a basic CRUD controller looks like:

```
class ListingController extends Controller
{
    public function index()
    {
        return Listing::query()
            ->published()
    }
}
```

```
        ->accepted()
        ->get();
    }

    public function show(Listing $listing)
    {
        if (!$listing->accepted || !$listing->published) {
            abort(404, 'Listing is not found');
        }

        return $listing;
    }

    public function store(Request $request)
    {
        return Listing::create($request->all());
    }

    public function update(Request $request, Listing $listing)
    {
        return $listing->fill($request->all());
    }

    public function destroy(Listing $listing)
    {
        $listing->delete();

        return response()->noContent();
    }
}
```

Perfectly standard so far. We're not even using custom requests or resources right now. The only rule is that we only list published and accepted listings.

These scopes are also pretty simple:

```

class Listing extends Model
{
    use HasFactory;

    protected $guarded = [];

    public function scopePublished(Builder $query)
    {
        $query->where('published', true);
    }

    public function scopeAccepted(Builder $query)
    {
        $query->where('accepted', true);
    }
}

```

In an application like this, you'll probably use these statuses a lot, so it's a good idea to have scopes like these ones.

Publish and accept

The next step is to implement the publish and accept functionality:

```

class ListingController extends Controller
{
    public function publish(Listing $listing)
    {
        $listing->publish();

        return response()->noContent();
    }

    public function accept(Listing $listing)
    {
        $listing->accept();

        return response()->noContent();
    }
}

```

```

    }
}

```

And the model looks like this:

```

class Listing extends Model
{
    use HasFactory;

    protected $guarded = [];

    /**
     * @throws Exception
     */
    public function publish(): void
    {
        if (!$this->accepted) {
            throw new Exception('Listing is not accepted yet');
        }

        $this->publish_at = now();

        $this->published = true;

        $this->save();
    }

    public function accept(): void
    {
        $this->accepted = true;

        $this->accepted_at = now();

        if (!$this->publish_at || $this->publish_at->isPast()) {
            $this->publish();
        }
    }
}

```

```

    $this->save();
}
}

```

Of course, there are two different roles here:

- Users can publish listings
- But only admins can accept them

For the sake of this example, I left these roles out. We want to learn about architectures, not permission handling or building a separate admin application.

Scheduling

The last step is to implement the scheduled publishing of the listings. This means we need a job that is triggered every minute by the `Console/Kernel` class. By the way, it can also be a command, but in my opinion, it's a good practice to execute these background tasks on separate worker servers. However, in this example, we don't deal with servers.

So this is what the job looks like:

```

class PublishListingsJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function handle(): void
    {
        Listing::query()
            ->shouldPublish()
            ->get()
            ->each
            ->publish();
    }
}

```

It queries listings that should be published and then publishes them. This is the `shouldPublish` scope:

```
public function scopeShouldPublish(Builder $query)
{
    $query
        ->accepted()
        ->where('published', false)
        ->where('publish_at', '<=', now());
}
```

Now let's try to summarize the advantages and disadvantages of this architecture. First, the advantages:

- It's really really simple and easy to understand.
- No overengineering and overcomplicated class structures/patterns.
- Easy to onboard new developers.

I think one of the most important advantages of this is that we can write code such as:

```
$listing->publish();
```

It feels pretty natural to use methods like this. We have an object that does something. Each model does a few tasks.

Unfortunately, the real world is more complicated than `$listing->publish()` so here are a few disadvantages:

- Models get ugly pretty fast. In this example, I have only two (!) features outside of a basic CRUD but the `Listing` model is already 65 lines long. I know it's not a lot, but it's only two (!) stupidly simple features. And this model doesn't even have a single relation, there is no user management in the app, etc.
- It's easy to mix responsibilities between models. For example, if I want to write a method that returns every listing that is older than 30 days for a single user where do I write this? In the `Listing` or the `User` model? I guess, now you say "Oh man, it's obvious! You need to write a `getOldListings` method in the `User` model. That's easy." It looks like this:

```
class User
{
    public function listings(): HasMany
    {
        return $this->hasMany(Listing::class);
    }
}
```

```

/**
 * @return Collection<Listing>
 */
public function getOldListings(): Collection
{
    return $this->listings()
        ->published()
        ->accepted()
        ->where('publish_at', '<=', now()->subMonth())
        ->get();
}
}

```

Not bad at all. The usage looks even better: `Auth::user()->getOldListings()` The only problem with this approach is that the `User` will get ugly really really fast. Just imagine a "bigger" application you worked on. How many relations did the `User` model have? 10? 25? 100? Just imagine a few methods like the `getOldListings` for every relationship. And also, in this case, we're writing a `Listing` query in the `User` class.

The other option is to write the query in the `Listing` model:

```

class Listing
{
    public static function getOldListings(User $user)
    {
        return $user->listings()
            ->published()
            ->accepted()
            ->where('publish_at', '<=', now()->subMonth())
            ->get();
    }
}

```

It's also not a bad solution, however, the usage feels a bit weird:

```
Listing::getOldListings(Auth::user())
```

As you can see, neither solution is perfect, and there's no right or wrong choice in my opinion. Let's continue with the disadvantages.

- It's easy to mix responsibilities between different classes. If you take another look at the `PublishListingsJob` it has business logic in it:

```
class PublishListingsJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function handle(): void
    {
        Listing::query()
            ->shouldPublish()
            ->get()
            ->each
            ->publish();
    }
}
```

Also if you take a look at the `ListingController` the `index` method looks like this:

```
public function index()
{
    return Listing::query()
        ->published()
        ->accepted()
        ->get();
}
```

Here's the problem with these kinds of methods:

- If you take the code and put it into the `Listing` model it'll have these weird, "one-time" functions that are usually not being reused at all. They are single-use-case functions. And it also leads to XL-sized models.
- If you leave them in controllers or jobs your project will be harder to understand, harder to maintain, etc. It'll be easier to duplicate code since developers should look for at least 3 classes to find a given query, for example. It's also harder to understand for new developers and/or juniors.

Invokable Controllers

To solve these problems we can use single action or invokable controllers. The idea is the following: each controller has only one method. Earlier we created a `ListingController` with the following functions:

- index
- show
- store
- update
- destroy
- publish
- accept

If we want to refactor this to invokable controllers it means that we'll have 7 different controllers:

- `GetListingsController` (index)
- `GetListingController` (show)
- `StoreListingController` (store)
- `UpdateListingController` (update)
- `DeleteListingController` (destroy)
- `PublishListingController` (publish)
- `AcceptListingController` (accept)

So each method has its own dedicated controller. In practice, I merged `store` and `update` together into a `UpsertListingController`. This is what the `GetListingsController` looks like, for example:

```
class GetListingsController extends Controller
{
    public function __invoke(Request $request)
    {
        return Listing::query()
            →published()
            →accepted()
            →get();
    }
}
```

This approach can be good since we can write much smaller controllers than before. The disadvantage is that we'll end up with a lot of small classes. However, in my opinion, it's a better situation than having fewer, but much bigger controllers.

But we still have the same problems as before. Responsibility is mixed between different classes. We still have business logic in jobs, controllers, and models. Controllers still cannot be reused from commands or jobs.

Services

A service is a pretty simple class that implements some business logic. It doesn't care about the "environment" so it doesn't matter if we call a method from a controller or a job. It just does something with a `Listing` for example. It means it cannot have arguments such as `Request` or `Resource`. These are environment (HTTP) dependent classes.

In our example, we can move every business logic to a class called `ListingService` that implements the basic CRUD functionality and other methods. Essentially, we're moving code from controllers to a service class.

This is what the `ListingService` class looks like:

```
class ListingService
{
    /**
     * @return Collection<Listing>
     */
    public function getAll(): Collection
    {
        return Listing::query()
            →published()
            →accepted()
            →get();
    }

    public function upsert(array $data, Listing $listing = null): Listing
    {
        return Listing::updateOrCreate(
            ['id' => $listing?->id],
            $data,
        );
    }

    public function delete(Listing $listing): void
    {
        $listing->delete();
    }

    public function accept(Listing $listing): void
```

```

{
    $listing->accepted = true;

    $listing->accepted_at = now();

    if (!$listing->publish_at || $listing->publish_at->isPast()) {
        $listing->publish();
    }

    $listing->save();
}

public function publish(Listing $listing): void
{
    $listing->publish();
}
}

```

As you can see, I just moved all the logic from the controller into this new class. Now the controller looks like this:

```

class ListingController
{
    public function __construct(private readonly ListingService $listingService)
    {
    }

    public function index()
    {
        return $this->listingService->getAll();
    }

    public function show(Listing $listing)
    {
        if (!$listing->accepted || !$listing->published) {
            abort(404, 'Listing is not found');
        }
    }
}

```

```
        return $listing;
    }

    public function store(Request $request)
    {
        return $this->listingService->upsert($request->all());
    }

    public function update(Request $request, Listing $listing)
    {
        return $this->listingService->upsert($request->all(), $listing);
    }

    public function destroy(Listing $listing)
    {
        $this->listingService->delete($listing);

        return response()->noContent();
    }

    public function publish(Listing $listing)
    {
        try {
            $listing->publish();
        } catch (CannotPublishListingException $ex) {
            abort(422, $ex->getMessage());
        }

        return response()->noContent();
    }

    public function accept(Listing $listing)
    {
        $this->listingService->accept($listing);

        return response()->noContent();
    }
}
```

```

    }
}

```

It doesn't seem like a big win, but here's a significant advantage: now the controller does what it's supposed to do. **It handles requests and responses.** It only cares about HTTP-related stuff such as requests, response codes, and things like that. Now every business logic is independent of the transportation layer (such as HTTP or CLI) and can be reused from anywhere.

If we need a console command that creates a new listing, all we need to do is this:

```

class CreateListingCommand extends Command
{
    public function handle(ListingService $listingService)
    {
        $data = [
            // Gather the command arguments
        ];

        $listingService->upsert($data);
    }
}

```

Here are the most important "rules":

- A controller should only contain HTTP-related code. This class is part of the "transportation layer."
- A command should only contain CLI-related code. This class is part of the "transportation layer."
- A service should only contain business logic. This class is part of the "business logic layer."

Of course, you can mix invocable controllers with services if you'd like to.

We still have one problem though: we still mix responsibilities between classes. The `Listing` model still implements the `publish` method because it's being used both in the `ListingService` and the `PublishListingsJob`. So right now, we can find Listing-related actions in both the `ListingService` and the `Listing` class. In my opinion, only queries and scopes should be inside models. It's highly subjective, by the way. With the new `ListingService` we can refactor our mini-application in such a way.

We have a pretty straightforward solution: we move the `publish` method into the `ListingService` and use it from the job and also the controller.

First, we move the `publish` method:

```

class ListingService
{
    /**
     * @throws CannotPublishListingException
     */
    public function publish(Listing $listing): void
    {
        if (!$listing->accepted) {
            throw CannotPublishListingException::because(
                'Listing is not accepted yet'
            );
        }

        $listing->publish_at = now();

        $listing->published = true;

        $listing->save();
    }
}

```

Then we use it from the `PublishListingsJob`:

```

class PublishListingsJob implements ShouldQueue
{
    public function handle(ListingService $listingService): void
    {
        Listing::query()
            ->shouldPublish()
            ->get()
            ->each(fn (Listing $listing) => $listingService->publish($listing));
    }
}

```

And also use it in the `ListingController`:

```

public function publish(Listing $listing)
{
    try {
        $this->listingService->publish($listing);
    } catch (CannotPublishListingException $ex) {
        abort(422, $ex->getMessage());
    }

    return response()->noContent();
}

```

With this, we just got rid of every "action-like" method from the model so now we have only queries (scopes in this case):

```

class Listing extends Model
{
    use HasFactory;

    protected $guarded = [];

    public function scopePublished(Builder $query)
    {
        $query->where('published', true);
    }

    public function scopeAccepted(Builder $query)
    {
        $query->where('accepted', true);
    }

    public function scopeShouldPublish(Builder $query)
    {
        $query
            ->accepted()
            ->where('published', false)
            ->where('publish_at', '≤', now());
    }
}

```

```
}
```

But of course, we have some disadvantages as well:

- `$listing->publish()` is much more human than `$this->listingService->publish($listing)` so we lost some readability.
- Now, instead of a big `ListingController` we have a big `ListingService`. Every method is reusable and independent of its environment (HTTP vs CLI vs Job) but still, all we did was move the entire `ListingController` into the `ListingService` class.

Still, if I need to choose between a big controller or a big service I'd go with the latter.

If you want to learn more about services (and repositories and actions) you can find an in-depth article [here](#).

And now, let's solve the service === controller problem!

Actions

If you think about it, using a service is almost identical to using a single controller but it has a number of advantages. Actions are pretty similar to invokable controllers but they also come with the advantages of service classes. Essentially, we can just copy the individual functions from the `ListingService` class and move them into small action classes. So instead of one `ListingService` class, we would have something like this:

- `GetListingsAction`
- `GetListingAction`
- `StoreListingAction`
- `UpdateListingAction`
- `DeleteListingAction`
- `PublishListingAction`
- `AcceptListingAction`

In practice, I merged `store` and `update` together into a `UpsertListingAction` class. This way we can get rid of large classes but still can have classes that are independent of everything and contain only business logic (no HTTP or CLI-related code).

Let's take a look at the `PublishListingAction` class:

```
namespace App\Actions;

class PublishListingAction
{
    /**
     * @throws CannotPublishListingException
     */
    public function __invoke(Listing $listing): void
    {
        if (!$listing->accepted) {
            throw CannotPublishListingException::because(
                'Listing is not accepted yet'
            );
        }

        $listing->publish_at = now();

        $listing->published = true;
```

```

        $listing->save();
    }
}

```

As you can see, it's an invokable class, which means we can do this in the controller:

```

public function publish(Listing $listing, PublishListingAction
$publishListing)
{
    try {
        $publishListing($listing);
    } catch (CannotPublishListingException $ex) {
        abort(422, $ex->getMessage());
    }

    return response()->noContent();
}

```

We inject the class into the `publish` method and then use it as if it was a function:

`$publishListing($listing)` I think it's a great syntax, however, it's not the only option. We can write a standard class with an `execute` function:

```

class PublishListingAction
{
    public function execute(Listing $listing): void
    {
        // ...
    }
}

```

And then use it as any other class:

```

public function publish(
    Listing $listing,
    PublishListingAction $publishListing
) {
    try {
        $publishListing->execute($listing);
    } catch (CannotPublishListingException $ex) {
        abort(422, $ex->getMessage());
    }

    return response()->noContent();
}

```

It's completely up to you which one you want to use. I use both of these techniques (but I'm always consistent in a given project).

Of course, actions can be embedded into one another. For example, the `AcceptListingAction` uses the `PublishListingAction`:

```

class AcceptListingAction
{
    public function __construct(
        private readonly PublishListingAction $publishListing
    ) {}

    /**
     * @throws \App\Exceptions\Listing\CannotPublishListingException
     */
    public function __invoke(Listing $listing): void
    {
        $listing->accepted = true;

        $listing->accepted_at = now();

        if (!$listing->publish_at || $listing->publish_at->isPast()) {
            ($this->publishListing)($listing);
        }
    }
}

```

```

        $listing->save();
    }
}

```

Since every action implements one thing (a unit of work) they can work together as if they were methods in a larger class. The `AcceptListingAction` class accepts the `PublishListingAction` class in the constructor which shows us the only disadvantage of an invokable action. I'm talking about this syntax:

```

($this->publishListing)($listing);

```

It's a bit weird, especially compared to this: `$this->acceptListing->execute($listing)`

Of course, these actions work from anywhere so we can use them in a Job as well:

```

class PublishListingsJob implements ShouldQueue
{
    use Dispatchable, InteractsWithQueue, Queueable, SerializesModels;

    public function handle(PublishListingAction $publishListing): void
    {
        Listing::query()
            ->shouldPublish()
            ->get()
            ->each(fn (Listing $listing) => $publishListing($listing));
    }
}

```

Or we can easily write a command that accepts a given listing:

```

class AcceptListingCommand extends Command
{
    protected $signature = 'listing:accept {listing}';

    protected $description = 'Accept a listing';

    public function handle(AcceptListingAction $acceptListing): void
    {
        $listing = Listing::findOrFail($this->argument('listing'));

        $acceptListing($listing);
    }
}

```

So as you can see, each action is literally a method from the `ListingService` discussed earlier. Is there a big difference between the two solutions? Not really, but here are some important points:

- Actions feel **cool**. Yep. And this is important. If you're an Apple fan, does it make you more productive/effective to type on a MacBook keyboard? Nope. But it feels at least 30% better compared to anything else, right? And it makes you a happier dev, and by the way, you probably feel more productive. Which is pretty important, in my opinion. Using actions gives you a similar feeling. (I'm an Apple fanboy myself so I can say it doesn't make me more productive but sure I feel like I am!)
- You're almost guaranteed to have only **smaller classes**. Or at least it's significantly harder to end up with 1000 lines long actions. It takes an incredibly complex system to have **a single** operation (such as creating a listing) bigger than 1000 lines of code. But if you write controllers with 10+ methods or a single service it's much easier to end up with huge (and ugly) classes.
- It's harder to **duplicate code**. You have a lot of small classes with pretty well-defined names and hopefully in clear namespaces. If you're looking for something in the project (for example, how can I accept a listing?) you know exactly where to look.
- Easier to navigate/onboard in your project. "Hey, new developer! So go to the `app/Actions/Listing` namespace and check out the classes. This is the entire feature set of our application when it comes to listings." It's basically a 1-1 representation (or close to 1-1) of user stories.
- It's getting more and more popular in the Laravel community. It means, at some point in the future, "action" will be a well-known concept in Laravel projects.

But of course there's no free lunch:

- Potentially, you'll end up with a huge number of classes. Of course, it depends on your project's size but you'll have lots of small classes. It gets pretty annoying when you have classes such as:
 - `PublishListingAction`
 - `PublishListingsJob`

- `PublishListingsAction`
- `UnpublishListingAction`
- etc
- Circular dependencies. It's possible that an action uses another action that uses the first one. Something like that:
 - Action A calls Action B
 - Action B calls Action A

I mean, it's a rare problem in my experience, but when you have 100s of small classes it's easier to have a circular dependency. If something like that happens it's probably because you have some problem in the underlying business logic so the fix can be harder than it looks (this is similar to when HTTP-based services call each other but in this case, the whole world ends).

Overall, actions are great! My favorite option in a number of situations. Here are some situations when you probably won't need them:

- You're indie hacking your first SaaS product. I think in this case, you should forget about actions, forget about services. Write everything in controllers and ship it.
- "Small" projects. Actions can be a bit of an overkill when you have only CRUD functions and a "small" number of models. Unfortunately, nobody ever defined what "small" is and I couldn't either so it's up to you to decide for yourself.
- Reusability is not that important. For example, you don't have commands or jobs only a simple API. Combine that with a "smaller" project and you're probably good to go with only controllers and models!

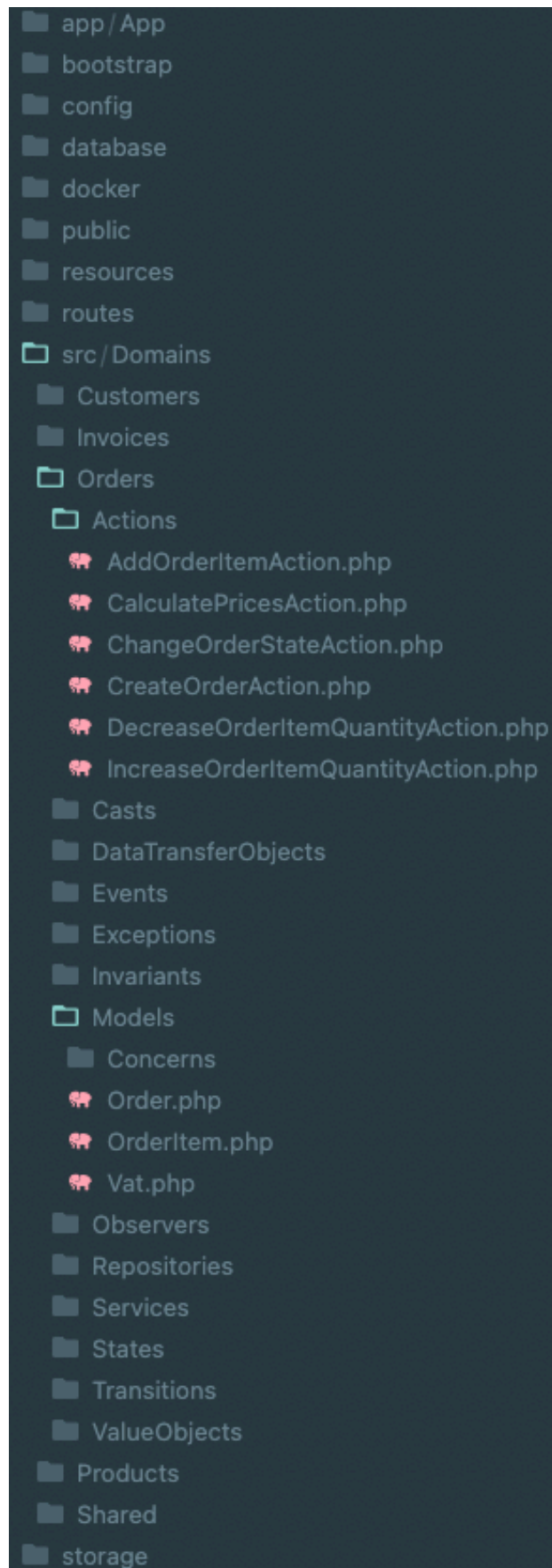
Do you remember when we couldn't decide that the `getOldListings` method should be in the `User` or `Listing` model? When we're using an action it's not a question anymore. It's gonna be a separate class:

```
class GetOldListingsByUserAction
{
    public function __invoke(User $user)
    {
        return $user->listings()
            ->published()
            ->accepted()
            ->where('publish_at', '<=', now()->subMonth())
            ->get();
    }
}
```

If you'd like to learn more about actions you can read my in-depth article [here](#).

Domains or modules

Instead of giving you a definition here's a screenshot:



So by "domain" or "module" what I mean really is just a folder with the name of your module. In the above image, you can see folders such as:

- Customers
- Invoices
- Orders
- Products

These are the main "modules" or the main "epics" of the application so we create a folder for each one. And then each of these domains contains the regular Laravel folders such as:

- Models
- Events
- Actions
- etc

This way, we have one folder that contains everything related to a domain. It's a great win in a "bigger" application in my opinion. These folders live inside an `src/Domains` folder and the good news is that you don't have to change anything related to Laravel's bootstrap process to make this work. You only need to change composer's autoload:

```
"autoload": {  
    "psr-4": {  
        "App\\": "app/",  
        "Domains\\": "src/Domains/",  
    }  
}
```

The important thing is: if you decide to go with some kind of module structure, **don't change anything Laravel-related** (such as `bootstrap.php`, `app.php`, `configs`, and so on). You'll probably regret it when a new version comes out and you try to upgrade your project.

If you take another look at the screenshot above you maybe noticed that there are no controllers or migrations in the domain folder. There's another concept called "applications". If you want to learn more about this topic you can read my in-depth article about [domains and applications](#).

DataTransferObjects (DTOs)

If you check out the `UpsertListingAction` class you see this:

```
class UpsertListingAction
{
    public function __invoke(array $data, Listing $listing = null): Listing
    {
        return Listing::updateOrCreate(
            ['id' => $listing->id],
            $data,
        );
    }
}
```

It's a pretty simple class, but what the hack is in the `$data` array? We don't know exactly. We need to look at the database columns or the FE to find out. It's not optimal. Wouldn't it be great if we had something like that:

```
class UpsertListingAction
{
    public function __invoke(
        ListingData $data,
        Listing $listing = null
    ): Listing {
        return Listing::updateOrCreate(
            ['id' => $listing->id],
            $data->toArray(),
        );
    }
}
```

This is called a data transfer object or DTO for short. It's a pretty simple class with only one responsibility: store some data and transfer it. That's it! Here's what the ListingData DTO looks like:

```
class ListingData
{

```

```

public function __construct(
    public readonly string $title,
    public readonly string $location,
    public readonly int $price,
    public readonly Carbon $publish_at,
) {}

public static function fromRequest(Request $request): self
{
    return new static(
        title: $request->title,
        location: $request->location,
        price: $request->price,
        publish_at: $request->publish_at ?? Carbon::parse($request-
>publish_at),
    );
}

public function toArray(): array
{
    return [
        'title' => $this->title,
        'location' => $this->location,
        'price' => $this->price,
        'publish_at' => $this->publish_at,
    ];
}
}

```

I think these classes can boost the readability and maintainability of your project significantly. However, this is not strictly related to architecture I just wanted to mention them. Of course, you can read more about them:

- [DTOs in more detail](#)
- [Actions and DTOs together](#)
- [Value objects](#) (people often confuse them with DTOs)

Further reading

Architecture is a huge topic and I love it! This is why I published two books on the topic:

- [Domain-Driven Design with Laravel](#)
- [Microservices with Laravel](#)

If you're interested in these books use the coupon code **l2a0rc33h** to get a 30% discount!